

1. How is RAG different from fine-tuning? Give a couple of scenarios where one is better than the other. [4 points]

RAG or Retrieval Augmented Generation is made up of two computer science concepts combined:

- Information Retrieval
- Generative AI

The idea is to integrate a general purpose large foundational model with a source of non-parametric knowledge which can easily be updated with latest data and it can store proprietary organizational data. Having such a mechanism provides a way to basically ground LLM responses and in turn reduce hallucinations.

While fine-tuning is done on domain specific stable knowledge and could be more expensive, RAG is more useful when you have frequently changing data like stock market data, or news headlines etc. The nature of this data is so dynamic that it doesn't make to internalize this data and hence you use RAG over fine-tuning.

Use Case	Nature of Data	Fine-Tuning	RAG
UW wants a chatbot to address student queries regarding admissions, events, departments etc	Frequently changing data	Not needed	Yes, RAG is a better system here which allows us to simply plug in and out frequently changing data and UW policies. This data could also be proprietary.
iSchool wants to build a general-purpose chat assistant that is good at creating technical notes from class transcripts	Here, the structure of notes (to be created) is very important	SFT that takes similar transcripts and their notes	Not needed as transcript will be uploaded
UW has a lot of events happening on-campus and it is difficult to keep track of all of them (Departments work in Silos sadly).	This is varying and changing data	Not needed	RAG is a better option where we first compile details of events from different sources on-campus and then make vector DB and update it regularly. LLM chatbot answers queries around events.

2. In the class we saw how to do generation using direct approach and RAG-based approach. Extend this in two ways: (i) either using a dynamic retrieval set (instead of a manually created one in the class) or using a larger model for generation, or both; and (ii) evaluating and comparing the outcomes from direct approach and RAG-based approach using appropriate metrics. Report how you extended the class code and your evaluation results. Do you see any advantages or costs of RAG? [6 points]

PART1: I used Langchain to implement a sequential pipeline where:

Instead of using a manually created knowledge base, I dynamically pulled relevant academic papers from arXiv using **ArxivLoader** based on the query **"RAG vs Fine-tuning"**. These documents were then split into chunks using **RecursiveCharacterTextSplitter** and stored in a **FAISS vector database** with **OpenAI embeddings**. Used ChatOpenAI model to generate answers (without RAG and with RAG)

I then set up a retriever with **search_type="mmr"** to ensure **diverse document retrieval**. Finally, I built a **LangChain pipeline** that combines the retrieved context with the input question and passes both to the language model for answer generation.

```
[117] main_chain = parallel_chain | prompt | llm # Connecting different segments

[121] main_chain.invoke('Compare RAG and RAFT?').content

' RAG (Retrieval-Augmented Generation) and RAFT (Retrieval-Augmented Fine-Tuning)
accompanied by both relevant ("golden") and irrelevant documents. The RAFT loss
emphasizes fine-tuning the model using curated datasets to improve performance

[122] my_ques = ['Should I use RAG or RAFT?',
                'Is Fine-tuning better than RAG?',
                'How RAFT works?']

[133] llm_responses = []
rag_responses = []

[134] for ques in my_ques:
    ans = llm.invoke(ques)
    llm_responses.append(str(ans.content))

    rag_response.append(str(main_chain.invoke(ques).content))

i = 0
for ques in my_ques:
    print(ques)
    print()
    print('---LLM Response---')
    print(llm_responses[i])
    print()
    print('---RAG Response---')
    print(rag_response[i])
    print('\n-----\n')
    i+=1

Should I use RAG or RAFT?

---LLM Response---
The choice between RAG (Retrieval-Augmented Generation) and RAFT (Reinforced AC

### RAG (Retrieval-Augmented Generation)
- **Purpose**: RAG combines retrieval mechanisms with generative models to enha
- **Use Cases**: Ideal for tasks where context or factual accuracy is crucial.
```

I found that RAG consistently produced **more accurate and grounded responses** across all three test questions. The direct LLM model, while fluent, showed some issues:

- In some cases, especially when the query involved less common terms like “RAFT” (intended as RAG vs Fine-Tuning), the vanilla LLM **misinterpreted the question entirely**, providing irrelevant or off-topic answers.
- The LLM also tended to give **generic explanations**, lacking the depth or factual grounding seen in RAG-generated responses.

On the other hand, the RAG approach, which used real-time retrieval from academic papers via arXiv, was able to:

- Offer richer context and detail
- Avoid hallucinations

Part 2: Evaluation:

To compare the effectiveness of direct LLM-based generation vs RAG-based generation, I used five questions related to the topic of RAG and RAFT. The responses were evaluated using BLEU, ROUGE-L, and semantic similarity scores.

	Question	BLEU (LLM vs RAG)	ROUGE-L (LLM vs RAG)	Semantic Similarity
0	Should I use RAG or RAFT?	0.000	0.000	0.032
1	Is Fine-tuning better than RAG?	0.000	0.000	0.019
2	How RAFT works?	0.002	0.077	0.334
3	What is the formula of loss function of RAFT?	0.008	0.096	0.589
4	Differentiate between PORAG vs RAFT	0.003	0.099	0.279

Extremely Low BLEU & ROUGE Scores:

Across all questions, the BLEU and ROUGE-L scores were close to zero, indicating that the vanilla LLM responses had little to no lexical overlap with the RAG responses. This was not surprising because the direct LLM hallucinated significantly, especially when it misunderstood "RAFT" as unrelated technical terms (e.g., optical flow, distributed systems, etc.).

Semantic Similarity Was Also Low:

The semantic similarity scores were also well below 0.6 for all but one question, confirming that the LLM misunderstood the queries or generated unrelated content in most cases.

So, in all, I found the vanilla LLM model struggling with disambiguating the term "RAFT", leading to factual errors and off-topic explanations. In contrast, the RAG either correctly answered or said 'I don't know' when information was lacking. This demonstrates a core benefit of RAG: its ability to ground responses and reducing hallucinations. Very very useful in fields like finance, healthcare and legal where explainability is important.